

백엔드에서 쌓은 판단을 AI Native 개발 방식으로 구조화합니다

Java·Kotlin으로 이벤트·배치·검색·캐시 시스템을 설계·운영했습니다. 이때의 판단을 에이전트·스킬·평가에 담아 개발 흐름에서 재사용합니다.

서민재 · Rex Seo

AI-Native Backend Engineer

ckdekrn88@gmail.com

GitHub

Velog

LinkedIn

이력서

경력기술서

01

POINT PLATFORM · EVENT RELIABILITY

발행 실패와 처리 실패에 서로 다른 복구 경로를 적용했습니다.

이벤트 레코드에 실패 지점과 복구 대상을 남겨 멈춘 업무부터 재실행했습니다. 포인트 차감·회수 실패 때 로그를 처음부터 추적할 필요가 없습니다.

담당 이벤트 상태·수동 복구·정체 알림 설계 및 구현	구조 내부 이벤트 9종, 상태 6개, 타입별 복구 전략 8개	저장 PostgreSQL internal_event_record · JSONB payload	관측 IN_PROGRESS 15분 초과 건을 타입별로 집계해 Slack 전송
상황 단일 ‘실패’ 상태: 원인 탐색부터 재시작 - 발행 실패·소비자 처리 중단: 확인 데이터·재시도 위치가 다름 - 단일 상태: 운영자가 로그·원장을 처음부터 수동 대조	판단 실패 지점 기록과 업무별 복구 로직 분리 - 상태 6개: 생성부터 복구 불가 실패까지 구분 - 포인트 차감·회수·만료 알림 등 작업 8개: 저장된 payload로 각 업무 서비스 재호출		

멈춘 위치에서 다시 시작하는 복구 흐름

생성부터 처리, 수동 복구와 정체 감지까지

호출 비동기 복구

01 정상 처리

업무 요청과 처리 상태를 기록한 뒤 Kafka로 전달합니다.

업무 요청

API · Admin · Batch

차감 · 회수 · 사용 · 만료

기록

상태 기록

internal_event_record
InternalEventCreator
CREATED · JSONB payload

전달

KAFKA

내부 이벤트 topic 9개

발행 실패는 SEND_FAILED로 분리

처리

업무 처리

Listener 9개 · EventProcessor

IN_PROGRESS → 완료 · 실패

02 수동 복구

Kafka에 다시 넣지 않고 실패한 업무만 실행합니다.

복구 대상 조회

type · status로 검색

저장된 payload 복원

선택

복구 전략

Retry Strategy 8개

이벤트 타입별 업무 분리

재실행

DOMAIN SERVICE

업무 서비스 직접 호출

차감 · 회수 · 사용 · 알림

03 정체 감지

15분 넘게 처리 중인 이벤트를 Slack으로 알립니다.

QUERYDSL

IN_PROGRESS 15분 초과

추적 대상 이벤트 8종

집계

SLACK

이벤트 타입별 요약

운영 채널로 건수 전송

달라진 운영

전송 실패와 처리 실패를 서로 다른 상태로 기록합니다. 처리 중 15분 넘게 멈춘 이벤트는 Slack으로 알리고, 저장된 payload로 실패한 업무만 다시 실행합니다.

세부 구현과 예외 처리

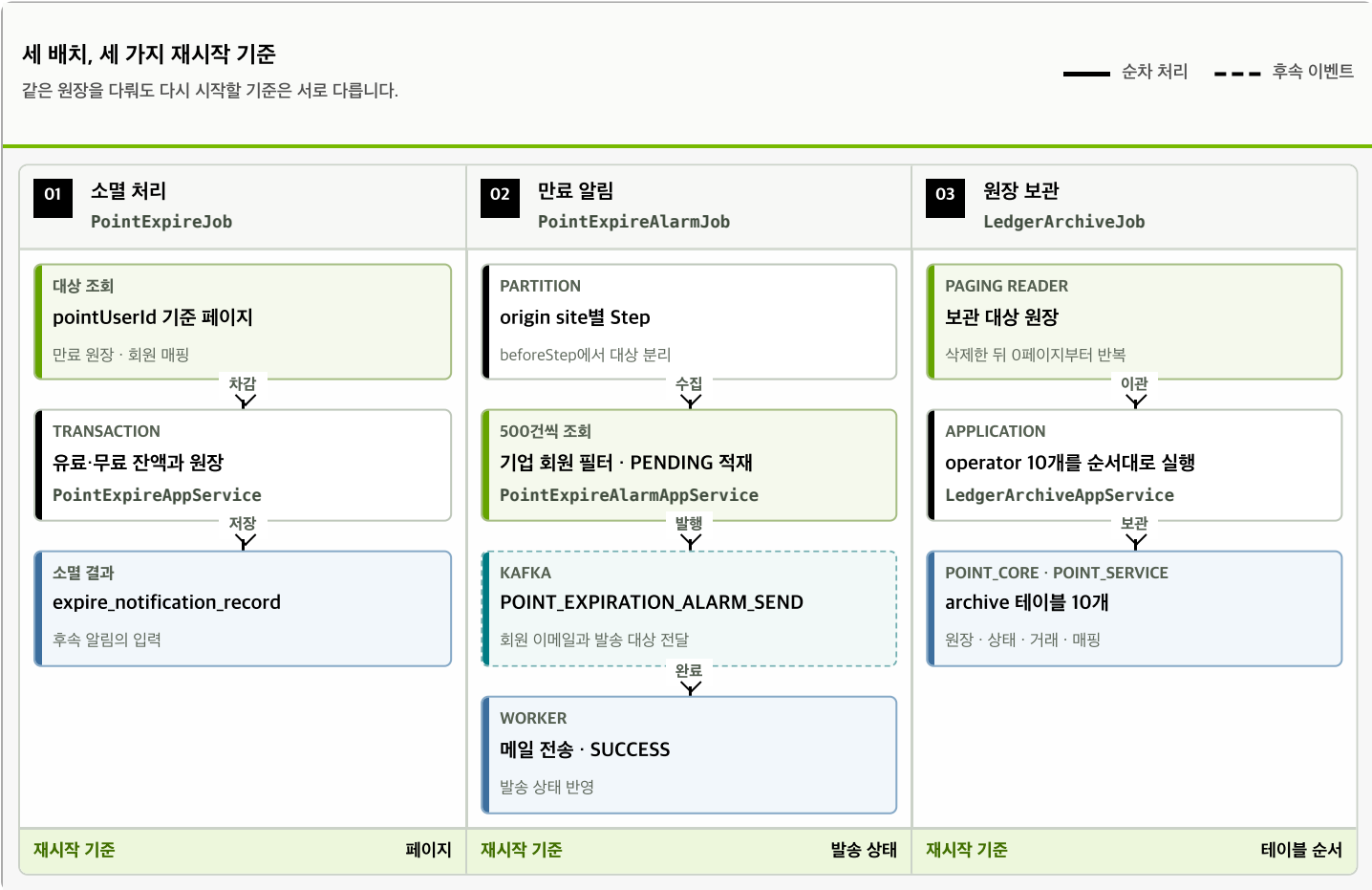
- 이벤트 상태 6개 내부 이벤트 9종의 생성·발행 실패·처리 중·완료·처리 실패·복구 불가 실패 기록
- 복구 전략 8개 수동 복구 API가 타입·상태로 레코드를 찾고 저장된 payload로 전략 선택·실행
- 실패 업무만 복구 Kafka 재투입 없이 타입별 업무 서비스를 직접 호출해 복구 범위 제한
- 15분 이상 정체 감지 IN_PROGRESS 추적 대상 8종만 조회해 타입별 건수를 Slack으로 전송
- 이벤트 결함 3종 상태 전환 타이밍·잘못된 상태 검사·발송 실패 로깅 누락 수정
- 재시도·복구 보강 배치 재시도·프록시 기반 재시도 호출 문제 수정, 포인트 선점 복구 경로 보강

배치는 멈춘 위치를, 동시 요청은 끝난 뒤의 상태를 확인했습니다.

원장 소멸·만료 알림·아카이브는 작업마다 다시 시작할 위치가 다릅니다. 포인트 거래와 예산 차감은 동시에 실행된 뒤 잔액과 원장 상태가 맞아야 합니다.

담당 원장 소멸 페이지 전환· 아카이브 배치 구현·만료 알림 이벤트 개선	처리 단위 전체 적재 대신 페이지 단위 반복 처리	보관 대상 point_core 6개·point_service 4개 archive 테이블	동시성 9개 API · 13개 시나리오 · 3·5·10개 동시 요청
--	--	---	--

상황 같은 데이터, 서로 다른 세 Job의 처리 기준 - 소멸: 잔액·원장 변경 - 만료 안내: 알림 적재·발송 - 아카이브: 참조 순서대로 테이블 이관 · 데이터 증가 시 전체 적재 실패 범위 확대	판단 배치는 재시작 위치, 동시 요청은 결과 불변식 - 소멸·알림·아카이브: 진행 위치 기록 - 포인트 거래·예산 차감: 락 획득 여부와 경쟁 종료 후 잔액·원장 상태·이벤트 수 모두 검증
--	--



락 키와 해제 시점에 따라 달라지는 세 가지 경쟁 결과

중복 차단·거래 정합성·어뷰징 분기의 세 경로로 나뉘었습니다.

9개 API 발급 · 거래 · 복구 · 예산 차감	13개 시나리오 같은 락 키 · 다른 거래 참조값 · 유지 시간 경계	3 · 5 · 10개 스레드 연속 재요청은 별도 타임라인으로 검증
--------------------------------	---	---

01 검증 대상

API마다 락 키와 해제 시점은 달라도 경쟁 후 상태는 설명할 수 있어야 합니다.

1회용 키 중복 발급 차단	포인트 적립 잔액 · 한도	포인트 선점 무료 · 유료 잔액	포인트 사용 원장 · 선점 상태	선점 복구 잔액 · 복구 상태
포인트 환불 원장 · 환불 상태	사용 중단 원장 · 중단 상태	사용 재개 원장 · 재개 상태	클릭 예산 차감 · 어뷰징 이벤트	

경쟁 후 검증 기준

성공 건수 · 잔액 · 상태 · 이벤트

02 중복 발급 억제 동일 사용자 · 락 유지 시간	03 거래 순서와 정합성 동일 사용자 · 서로 다른 거래 참조값	04 클릭 어뷰징 분기 복합 요청 식별자 · 락 유지 시간
10개 스레드 같은 사용자의 키 발급 요청 같은 식별자로 동시에 진입 차단 기대 결과 동시 요청 10건 · 생성된 키 1개 같은 사용자에게 키가 중복 생성되지 않음	3개 스레드 API별 서로 다른 거래 정보 3건 적립부터 사용 중단·재개까지 7개 거래 API 순차 처리 기대 결과 3건 모두 완료 성공 여부와 함께 상태를 함께 확인	5개 스레드 고유 요청과 중복 요청 복합 요청 식별자로 중복 여부를 구분 판별 기대 결과 고유 요청 5건 · 중복 요청 1건 처리 중복 4건은 대체 처리·어뷰징 이벤트로 전환
락 유지 시간 경계 경과 후 재요청 허용	불변식 잔액 · 원장 · 업무 상태	후속 처리 차감 · 대체 처리 · 이벤트
잔액 무료·유료 잔액과 적립 한도 누적액	원장·업무 상태 사용 · 복구 · 환불 · 중단 · 재개	후속 이벤트 차감 · 대체 처리 · 어뷰징 이벤트 수

CI 느린 테스트의 실행 경계

DisableSlowTest가 붙은 테스트는 기본 실행에 포함하고, skip.slow.test=true일 때 제외합니다.

실행 정책	기본 실행 · skip.slow.test=true 제외
-------	--------------------------------

정합성과 회원 식별 처리

- 만료 처리 순서 대상 수집→날짜 계산→기업 회원 필터→알림 레코드 일괄 저장 검증
- 동시성 경계 같은 락 키·서로 다른 거래 참조값 조합과 동일 예산의 동시 요청을 각각 검증
- 10개 테이블 아카이브 point_core 6개·point_service 4개 흐름과 테스트
- 혼합 원장 유료·무료 원장 혼합 사용 케이스 분리 검증
- 회원 식별 경계 회원-워크스페이스 매핑·master/member 분리·탈퇴 이벤트·globalUserRef 중복 처리 보장

FEED & SEARCH · SERVING / RETRIEVAL

03

후보 수보다, 검색·랭킹·서빙의 경계를 먼저 설계했습니다.

개인화 피드는 후보 수집·FPR·SPR·Re-Rank·Snapshot으로 경로를 나눴습니다. 랭킹은 pull 계약, OpenSearch 조회·Redis 캐시는 포트로 격리했습니다.

9개 검색 소스 후보 조회 포트	FPR · SPR · Re-Rank 공통 pull 계약	Snapshot · FPR · SPR 3종 캐시 포트
----------------------	-----------------------------------	----------------------------------

전체 흐름 Candidate Retrieval → FPR → SPR → Re-Rank → Snapshot	공통 계약 후보 수집 포트 · 역방향 pull 계약 · 캐시 포트 3개	검색 경계 검색 소스 9개 · 소스별 정책 · 타임아웃·실패 격리	모듈 경계 query-api는 OpenSearch 연동 전용 · 순환 의존 금지
--	---	--	--

상황 작은 정책 변경도 전체 피드 경로로 전파 • 변경 축: 후보 소스·랭킹 정책·캐시 수명 • 단일 오케스트레이터: 검색 쿼리·점수 계산·Redis 처리 결합	판단 오케스트레이터는 서빙 흐름만 조정 • 계산·캐시: 단계별 계약으로 분리 • Snapshot Empty: Re-Rank가 SPR→FPR→후보 수집 순으로 필요한 후보 pull • Redis 장애: 동일 랭킹 흐름 · 캐시 우회
--	---

개인화 피드 온라인 서빙의 책임 경계

상위 계층은 서빙 흐름을 조정하고, 계산과 캐시는 단계별 계약 뒤로 분리했습니다.

01 요청과 서빙 경로

HTTP 입력을 정규화하고 캐시 상태에 따라 서빙 흐름을 정합니다.

GET /V1/API/FEED
Feed API
HTTP · session header

정규화

CORE CONTRACT
FeedRequestContext
actor · cursor · filter

실행

ROUTE OWNER
Orchestrator
queue · refill · bypass

pop

REDIS LIST
Snapshot Queue
최종 window · page I/O

Snapshot 상태가 다음 경로를 결정합니다.

Hit = page · Empty = pull · Unavailable = cache off

02 역방향 pull 캐스케이드

최종 단계가 필요한 후보만 앞 단계에서 당겨옵니다.

EXPOSURE POLICY
Re-Rank
최종 노출 순서

pull

RANKING + POOL
SPR
정밀 점수 · SPR cache

pull

RANKING + POOL
FPR
dedupe · seen · FPR cache

pull

CORE PORT
Candidate Retriever
회원 9개 소스 · 비회원 인기 공고

I/O

EXTERNAL SEARCH
OpenSearch
query-api 어댑터 · 회원 소스 9개

데이터 흐름
OpenSearch → 후보 조회 → FPR → SPR → Re-Rank → Snapshot

pull 호출
Snapshot 소진 → Re-Rank → SPR → FPR → 후보 조회

Orchestrator
route · invalidation · page I/O

Ranking Stage
단계별 계산 · FPR/SPR pool

Query Adapter
OpenSearch I/O · 소스 실패 격리

바뀌는 이유가 다른 코드를 서로 떼어냈습니다.

캐시 상태에 따라 서빙 흐름과 페이지 입출력을 조정합니다. OpenSearch 필터·정렬·호출은 query-api, FPR·SPR·Re-Rank는 랭킹 단계, Snapshot·FPR·SPR 캐시는 포트로 분리했습니다.

랭킹·캐시·검색 정책 상세 • 후보·랭킹 경계 후보 수집은 별도 조회 포트, FPR·SPR·Re-Rank는 공통 pull 계약으로 연결 • 캐시 상태 계약 Snapshot·FPR·SPR 포트의 Hit·Empty·Unavailable 공통 표현 • 검색 소스 9개 격리 OpenSearch 소스별 필터·정렬·벡터 분리, 한 소스의 타임아웃·실패가 나머지 후보 조회에 미치는 영향 차단	• Redis 장애 우회 별도 랭킹 없이 동일 흐름을 캐시 없이 실행 • query-api 경계 OpenSearch 연동만 담당, 모듈 방향·순환 금지는 ArchUnit으로 검증 • 검색 문서 38개 필드 @SearchDocument 기반 프로세서로 경로 생성, 계약 테스트로 직렬화 결과·상수 일치 검증
--	--

- 04

테스트마다 반복하던 준비 코드를 test-support로 모았습니다.

테스트 진입점 4개와 공통 도구·정책 5개를 만들었습니다. 프로젝트마다 달랐던 fixture·mock 준비 방식을 통합하고 테스트 선택·CI 실행 기준을 문서화했습니다.

진입점, 생명주기, 컨텍스트 범위, 실행 정책을 한 기준으로 묶었습니다.

공통 기반		Mock 테스트		MVC 테스트		JPA 테스트	
01 · fixture·생명주기		02 · 협력 객체 격리		03 · HTTP 요청·응답 계약		04 · 매핑·쿼리·영속성	
BASE CLASS AbstractTest PER_CLASS · 공통 fixture		BASE CLASS AbstractMockTest mock 생성 · 초기화		BASE CLASS AbstractWebMvcTest @WebMvcTest · Controller 자동 등록		BASE CLASS AbstractDataBaseTest @DataJpaTest · Repository·Config 등록	
↓		↓		↓		↓	
FIXTURE FixtureMonkeyFactory fixture 생성 규칙		EXTENSION AutoMockExtension 주입 · reset · session 정리		WEB HELPER WrappedMockMvc 요청 · 응답 검증용 래퍼		DATABASE HELPER DB_INITIALIZER Repository·Config Bean 등록	

공통 도우미와 함께 테스트 진입점, mock·컨텍스트 생명주기, 별도 트랜잭션, 느린 테스트의 실행 기준을 문서화했습니다.

제공 기능과 CI 처리

- **테스트 진입점 4종** 기본·mock·WebMvc·JPA에 FixtureMonkey·mock 생명주기 연결
- **도우미 공통화** Controller·Repository 자동 등록·응답 검증·REQUIRES_NEW 트랜잭션
- **느린 테스트 분리** 테스트 작성 원칙 제안, 동시성 테스트를 skip.slow.test 설정으로 격리

에이전트·스킬을 설치 가능한 자산으로 묶고, 구조와 행동 계약을 나눠 검증했습니다.

코드 탐색·설계 검토·테스트 절차를 15개 에이전트·32개 스킬로 나눴습니다. 구조 케이스 7개·행동 케이스 10개·설치 회귀 52개로 의존성과 계약을 검증했습니다.

Agent가 역할을 정하고, skill이 절차를 제공하며, KB가 판단 근거를 보탭니다.

Agent Lab이 직접 작업을 실행하지는 않습니다. 자산과 의존성을 묶어 각 런타임에 설치합니다.

—— 실행 순서 - - - 런타임 경계

ASSET REGISTRY · V0.1.0.0

manifest.txt

선택한 자산과 연결된 의존성을 한곳에서 찾습니다.

AGENTS

15

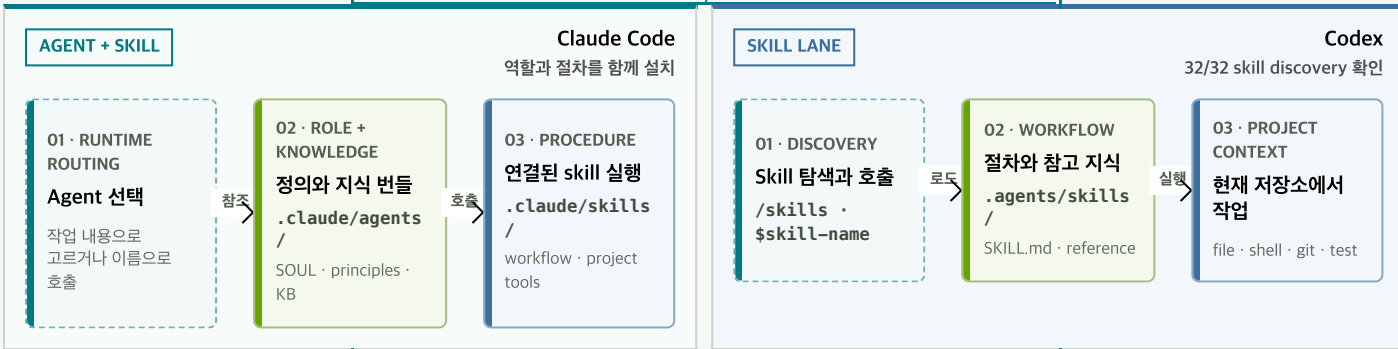
BUNDLES

14

SKILLS

32

설치된 자산을 런타임이 탐색·선택



SHARED EXECUTION

저장소 상태를 먼저 확인하고

코드·설정·도구를 기준으로 작업 범위를 정합니다.

OUTPUT CONTRACT

수정 근거와 검증 결과까지 남깁니다.

file:line · test result · risk · next action

- | | | | |
|---|--|---|--|
| 01
자산 선택
agent · skill · all-* | 02
연결 항목 확인
연결 skill을 재귀 탐색 | 03
런타임 경로에 설치
dry-run · apply · verify | 04
회귀 검사
설치 52/52 · Codex 32/32 |
|---|--|---|--|

OpenCode .opencode 경로 배치는 검사했습니다. agent 포맷 변환과 실제 선택·실행은 아직 검증하지 않았습니다.

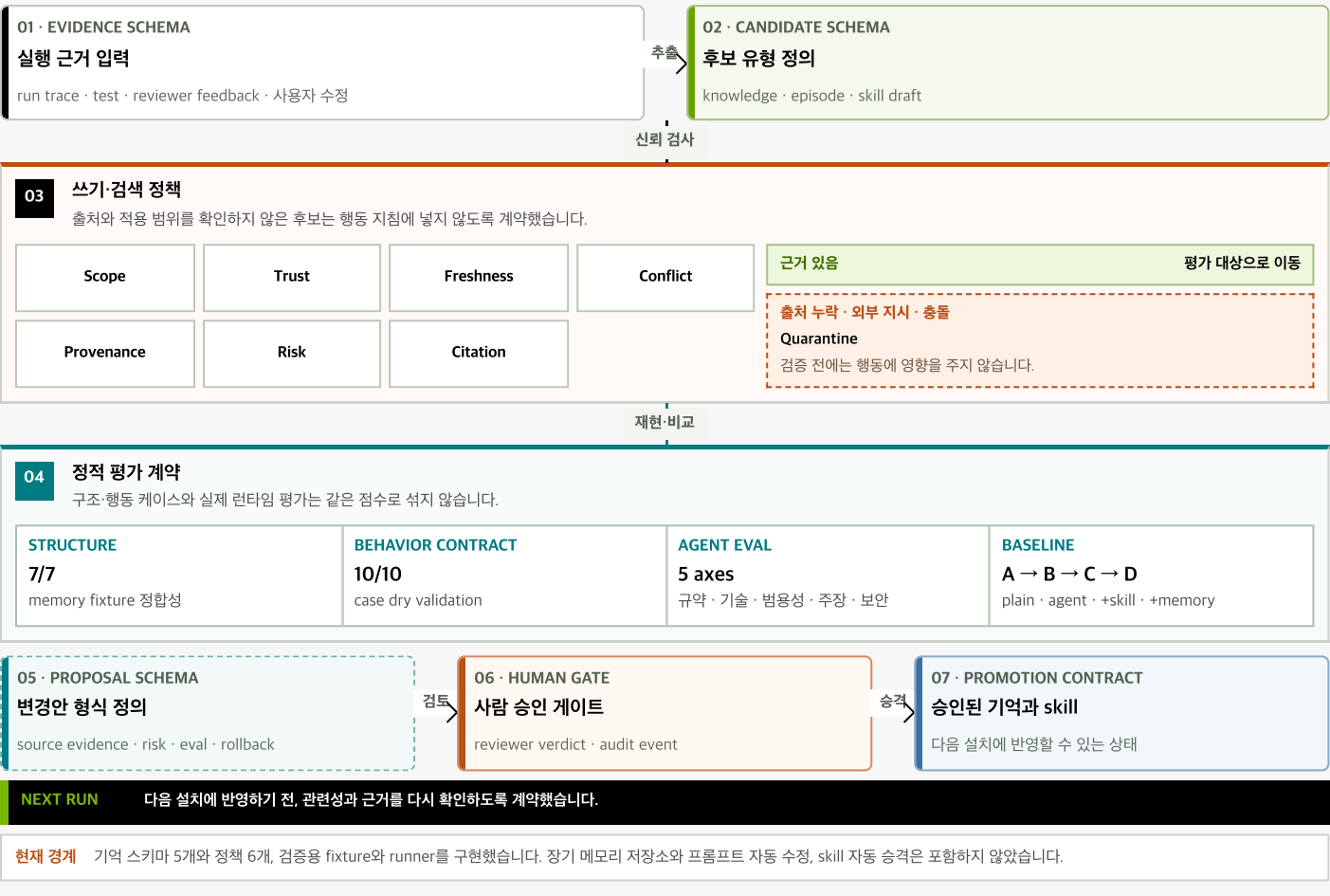
현재 설치 범위

Claude Code에는 에이전트·스킬을, Codex에는 스킬 32개를 설치할 수 있습니다. 메모리·자가 수정은 스키마·정책·평가 계약으로 경계만 정의하고 실제 반영에는 사람 승인을 요구합니다.

기억 후보를 검토·승인하기 위한 계약

후보가 실제 동작에 영향을 주기 전에 확인할 스키마·정책·평가 경계를 정의했습니다.

승격 후보 격리



테스트, 인증, 검색 경험이 지금의 설계 기준이 되었습니다.

실제 구현 테스트·인증·인가 경계·OpenSearch 동적 쿼리 경험을 이벤트 신뢰성, 배치 재시작, 피드 캐시·검색 정책의 설계 기준으로 삼았습니다.

토스랩 · JANDI

2024.06 — 2025.04

약 200만 건을 보정했습니다. Mock 테스트가 놓친 JPA 변경 감지 오류는 실제 구현 테스트로 재현했습니다.
Drive-Service를 Spring Boot 2.3.8→3.3.3으로 전환하며 테스트를 0→160개 이상으로 늘렸습니다. 커버리지 70%에도 불안했던 배포 원인은 테스트 기준·CI 실행 방식에서 찾았습니다.

오케스트로 · IDP / Vista

2023.09 — 2024.05

신용정보원 IDP에서 Spring Cloud API Gateway와 Spring Security·JWT 인증·인가, SonarQube·Jenkins 연동 API를 개발했습니다.
하나금융그룹 Vista에서는 VM 메트릭 조회 API와 OpenSearch 동적 쿼리 분기를 구현했습니다.

즐거운 · Backend

2022.11 — 2023.02

자사 서비스의 백엔드 기능을 개발하고 운영 중 발생한 문제를 해결했습니다.

구현 과정과 판단 근거를 글·발표·오픈소스로 공개했습니다.

기술 글 156편, YouthCon 발표, Spring OSS 개선 3건으로 테스트와 데이터 복구 경험을 공유했습니다.

YouthCon 2025 연사 ‘우리팀을 위한 Spring Test 만들기’에서 테스트 코드를 줄이는 기준과 팀의 그라운드 룰을 만드는 과정을 발표했습니다.	기술 글 156편 · 2026.07 기준 Java, JPA, 테스트, SQL 튜닝을 중심으로 시행착오를 기록했습니다. 글도 10기에서는 백엔드·인프라 빌리지 준비위원과 연사로 참여했습니다.
Spring OSS · 개선 제안 3건 반영 Kafka, Boot, Data JPA에 제안한 불변 컬렉션 처리·하드코딩 제거·자원 정리 로직 개선 3건이 반영됐습니다.	OpenSearch 한글 분석 플러그인 JavaCafe Elasticsearch 플러그인의 한·영 자판 변환과 초성·자모 검색을 OpenSearch 2.19·Kotlin으로 이식해 16개 예시로 검증했습니다.
화면에 없던 7만+ 건의 오류 S3에서는 삭제됐지만 DB에는 6년 동안 7만 건 넘게 쌓인 데이터를 발견했습니다. 응답이 정상인 상황에서 원인을 좁혀 간 과정을 공개했습니다.	테스트로 찾은 약 200만 건 Mock 테스트가 놓친 JPA 변경 감지 오류를 실제 구현 테스트로 재현했습니다. 보정 스크립트를 작성해 약 200만 건의 데이터를 바로잡았습니다.